Что мы сделали: полное объяснение

Проект про автоматический подбор признаков для рекомендательной системы на данных Яндекс.Музыки (Yambda-50M). Документ написан для того, чтобы вы могли разобраться в каждом шаге и рассказать о нём сами.

> **Итог в пяти строках.**
> 1. Мы собрали рекомендательную модель и обучили её на 5000 пользователей Яндекс.Музыки.
> 2. Модели с признаками на 40% лучше случайного порядка по NDCG@10 (около 0,082 против 0,0575 на тесте).
> 3. Автоматический генератор (151 признак) **не отличается от ручного набора (37 признаков) в пределах шума**.
> 4. Зато автоматика нашла то, что человек пропустил: самые полезные признаки считаются по окну в **180 дней**, а в ручном наборе окон длиннее 30 дней не было.
> 5. Утверждать, что автоматика обыграла хорошо затюненный ручной бустинг, **нельзя**: гиперпараметры мы не подбирали, а разницу между моделями шум перекрывает.

1. Что вообще требовалось

Когда приложение предлагает вам трек, за этим стоит модель, которая по таблице чисел предсказывает: понравится ли вам этот трек. Числа в таблице называются **признаками** (по-английски «фичи»). Например: «сколько раз вы слушали этого артиста за последнюю неделю».

В реальных сервисах признаки придумывают инженеры вручную: 10–100 штук. Задание ставит вопрос: **а вдруг человек выбрал не лучшие?** Он решил считать «лайки за 7 дней», но, может быть, «за 10 дней» работало бы лучше. Никто не проверял. Нужно было придумать способ, который подбирает такие вещи автоматически, и показать на цифрах, что он работает.

Что сдавать по заданию: рабочий пайплайн (модель с ручными признаками как «планка»), идею автоматизации и эксперименты, которые её проверяют, а также отчёт в читаемом виде. Этот PDF и репозиторий на GitHub это и есть отчёт.

2. Словарик (вернитесь сюда, если что-то непонятно)

Термин	Что это простыми словами
Признак (фича)	Число, описывающее ситуацию: «юзер слушал этого артиста 12 раз», «трек дослушивают в среднем на 80%».
Метка (label)	Правильный ответ, который модель учится угадывать. У нас: 1, если юзер лайкнул трек вскоре после прослушивания, иначе 0.
Градиентный бустинг (LightGBM)	Алгоритм: строит сотни маленьких деревьев решений, каждое исправляет ошибки предыдущих. Хорошо работает на таблицах.
Классификатор / ранкер	Классификатор предсказывает вероятность «лайк / не лайк» для каждой строки отдельно. Ранкер (LambdaRank) учится правильно упорядочивать кандидатов внутри одного юзера.
Окно (7 дней, 30 дней…)	Период в прошлом, за который считается признак. «Лайки за 7 дней» = лайки за неделю до момента прогноза.
Срез (cutoff)	Момент времени «сегодня» для модели. Всё до него модель знает, всё после ещё не произошло.

Термин	Что это простыми словами
Утечка данных	Когда в признаки случайно попадает информация из будущего. Модель тогда блестяще работает на проверке и бесполезна в жизни.
Train / val / test	Три куска данных по времени, идущие друг за другом. На train модель учится, на val мы выбираем настройки, на test проверяем итог.
AUC	Насколько хорошо модель отличает «лайк» от «не лайк» по всем строкам сразу. 0,5 = монетка, 1,0 = идеально.
NDCG@10	Для каждого юзера модель выстраивает кандидатов по убыванию вероятности лайка. NDCG@10 смотрит, насколько высоко в первой десятке оказались лайкнутые треки. Выше = лучше. Главная метрика в этом проекте.
Recall@10	Какая доля лайкнутых треков юзера попала в первую десятку. Выше = лучше.
Бейзлайн	Простейший способ для сравнения (случайный порядок). Если модель не лучше бейзлайна, она не нужна.
Bootstrap	Способ измерить шум. Много раз случайно пересэмплируем пользователей и смотрим, как меняется метрика. Получается интервал: «настоящая разница лежит примерно здесь».

3. Данные и подготовка

Мы взяли открытый датасет Яндекса **Yambda-50M** (прослушивания и лайки в Яндекс.Музыке) и скачали его с Hugging Face.

Файл	Что внутри	Размер
listens	Прослушивания: юзер, трек, время, «органика или рекомендация», доля прослушанного, длина трека	46,5 млн строк
likes	Лайки: юзер, трек, время, «органика или рекомендация»	881 тыс. строк
artist_item_mapping	Какому артисту принадлежит каждый трек	9,27 млн строк
album_item_mapping	В каком альбоме каждый трек	9,65 млн строк

Подвыборка

46 млн строк тяжело для ноутбука, поэтому мы случайно выбрали **5000 пользователей** и взяли **всю** их историю (а не случайные строки, иначе у каждого юзера история получилась бы рваной). Выбор зафиксирован числом seed=42: при повторном запуске выберутся те же пользователи.

Что мы предсказываем

Каждая строка в нашей таблице это одно **прослушивание**. Метка равна 1, если пользователь лайкнул этот трек в течение часа после начала прослушивания, иначе 0. Лайк это редкое событие: позитивов около 1%. Задача звучит так: «из треков, которые юзер слушал в ближайшие две недели, какие он лайкнет?» Модель ранжирует эти треки, и мы смотрим, оказались ли лайкнутые наверху.

> **Важная деталь.** Кандидаты у нас только те треки, которые юзер действительно слушал, а не весь каталог. Поэтому задача ближе к «переупорядочить то, что человек всё равно послушал», чем к «найти новое в каталоге». Это упрощение, и оно записано в ограничениях.

4. Как мы избегаем «подглядывания в будущее»

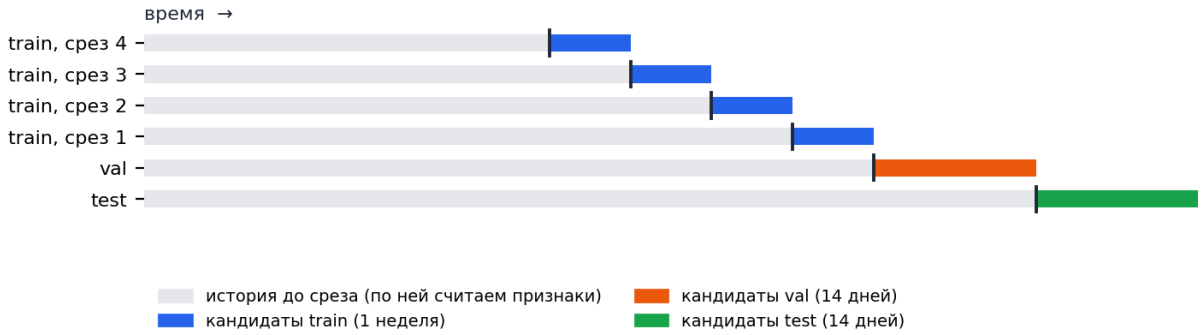
Самая частая ошибка в такой работе это **утечка данных**: модель случайно получает информацию из будущего и показывает отличные цифры, которые не повторятся в реальности. Мы защищались тремя способами.

4.1. Деление по времени

Данные делятся не случайно, а по времени: самые поздние 14 дней это test, предыдущие 14 дней это val, всё раньше это train. В жизни мы всегда предсказываем будущее по прошлому, и проверка должна быть такой же.

4.2. Схема «срез + окно»

Выбираем момент (срез). Признаки считаем **только** по событиям строго до среза. Кандидатов берём из периода после среза. Для train мы взяли 4 недельных среза подряд, чтобы обучающих примеров было больше. В коде стоят проверки (`assert`), которые падают, если история залезла за срез или кандидат оказался раньше среза.



Серая часть это история, по которой считаются признаки. Цветная часть это кандидаты, чьи лайки мы предсказываем. Чёрная черта это срез. У каждой выборки история заканчивается там, где начинаются кандидаты.

4.3. Фильтр «уже лайкнуто»

Из кандидатов мы выкинули пары «юзер + трек», которые юзер уже лайкал *до* среза: новым лайком они стать не могут. Об этом ниже, в разделе про найденную проблему.

5. Признаки: ручные и автоматические

5.1. Ручной набор («планка»)

Это то, что сделал бы обычный инженер. Признаки трёх видов:

Группа	Примеры	Зачем
Юзер (`u_`)	сколько прослушиваний и лайков у юзера за 1, 7, 30 дней и за всё время; доля лайков; средняя доля дослушивания	активность и вкус человека
Трек (`i_`)	то же самое по треку: популярность, доля лайков, как часто дослушивают	насколько трек «заходит» людям
Юзер × трек (`ui_`)	сколько раз юзер слушал именно этот трек	прямая связь пары
Юзер × артист / альбом (`uar_`, `ual_`)	сколько раз юзер слушал и лайкал треки этого артиста или альбома	косвенная связь: любит артиста, значит, вероятно, лайкнет и его трек

Набор без артистов и альбомов это 37 признаков (его мы называем **full manual**), с артистами и альбомами 61 признак. Окна в нём выбраны вручную: 1, 7, 30 дней и «всё время».

5.2. Автоматический генератор

Мы описали признак шаблоном **«сущность × событие × окно»** и заставили компьютер перебрать все комбинации:

- **5 сущностей:** юзер, трек, юзер×трек, юзер×артист, юзер×альбом.
- **2 события:** прослушивания и лайки.
- **10 окон:** 1, 3, 7, 14, 21, 30, 60, 90, 180 дней и «всё время».
- К счётчикам добавлены производные: доля лайков и средняя доля дослушивания.

Получилось **151 признак** вместо 37. Код лежит в файле `src/auto_features.py`. Мы проверили его на тестовых данных: на 64 признаках, общих с ручным кодом, значения совпали полностью.

5.3. Отбор лучших

151 признак это много, а ручной набор содержал 37. Чтобы сравнивать честно (при равном «бюджете»), мы оставляли 37 самых важных автоматических признаков. «Важность» берётся из обученной модели (показатель gain: сколько пользы признак принёс деревьям). Ещё пробовали версию, где похожие друг на друга признаки отсеиваются по корреляции.

6. Как мы оцениваем качество

Основная метрика **NDCG@10**. Она считается внутри каждого пользователя: модель ранжирует его кандидатов, и мы смотрим, где в списке оказались лайкнутые треки. Вторая метрика Recall@10. AUC мы тоже печатали, но ему нельзя слепо доверять: он считается по всем строкам сразу и сильно зависит от того, что одни юзеры лайкают много, а другие мало. Это не говорит, что модель умеет упорядочивать треки.

Бейзлайны: что считается «ничего не умеет»

random (случайный порядок) даёт NDCG@10 около 0,0575 на тесте. **top-popular** (ставить выше самые популярные треки) тоже около 0,055: общая популярность почти не помогает угадать личный лайк, потому что все кандидаты это треки, которые юзер и так слушал. Значит, весь полезный сигнал сидит в признаках про пару «юзер × трек», про самого юзера и про трек.

7. Что нашли по дороге (самое поучительное)

7.1. Слишком хорошая метрика оказалась артефактом

> Первая версия показала AUC 0,87 и NDCG 0,07–0,08. Для такой задачи это подозрительно хорошо. Причина (наиболее вероятная): если юзер лайкнул трек *раньше*, чем слушал его снова, то метка равна 0, и признак «юзер уже лайкал этот трек» становился подсказкой «ответ точно 0». Модель училась этому правилу, а не вкусу.

Что сделали: убрали из кандидатов все пары, где лайк уже был до среза. Теперь модель предсказывает именно **новые** лайки. Выборки уменьшились, а метрики пересчитаны, и цифры в этом документе уже после фильтра. Прямую проверку гипотезы мы не запускали, поэтому причина названа вероятной, но фильтр логичен сам по себе.

7.2. Разница между моделями тонет в шуме

На 2000 пользователях ручной набор «обыгрывал» упрощённый. Мы увеличили выборку до 5000 пользователей, и «преимущество» исчезло: оно оказалось шумом. Тогда мы начали измерять шум явно, через **bootstrap по пользователям**: 100 раз случайно пересэмплируем юзеров и смотрим, как меняется разница между моделями. Получается 95%-интервал. Если ноль внутри интервала, разницу нельзя отличить от шума. Типичная ширина интервала у нас около ±0,007 по NDCG@10. **Любая разница меньше этого порога не доказана.**

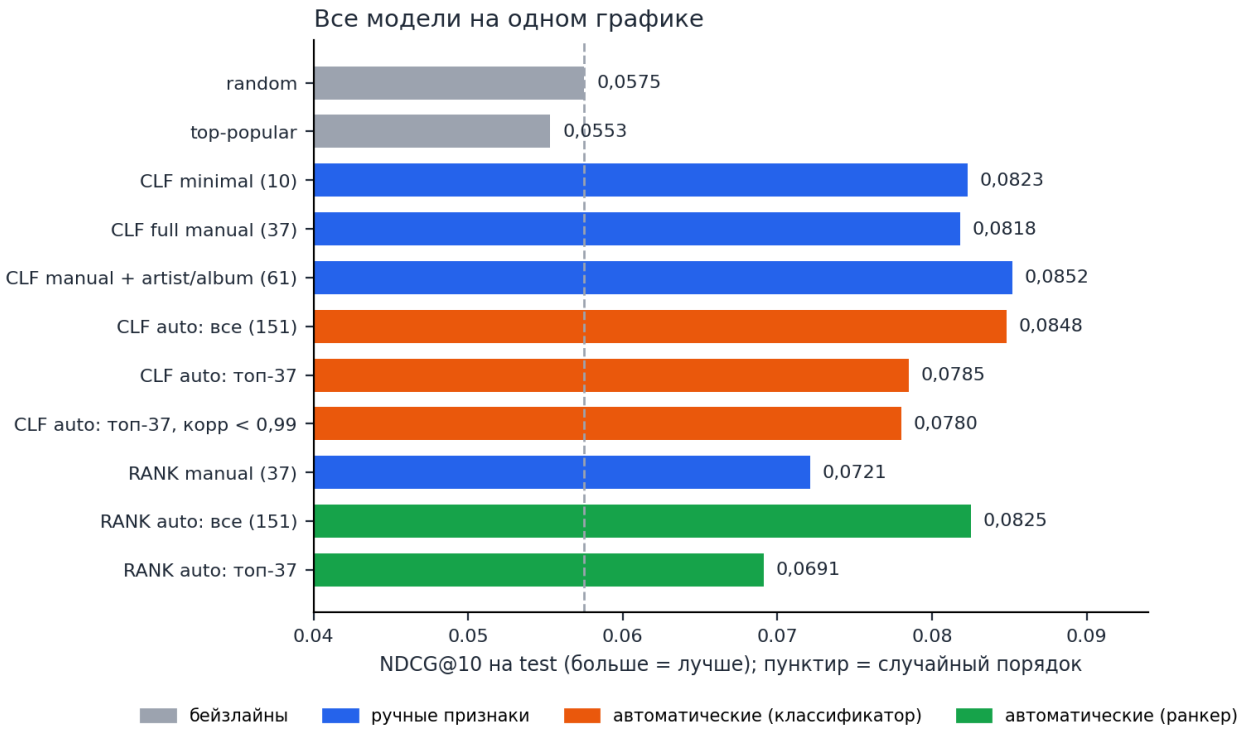
7.3. Что нужно обязательно запомнить

- Метрики на разных выборках (до и после фильтра, 2000 и 5000 юзеров) **нельзя сравнивать между собой**.
- Выбор варианта делается по val, а test смотрится для проверки. Иначе получится подгонка под test.
- Сравнивать нужно с **лучшей** ручной моделью, а не с удобной слабой.

8. Результаты

Все цифры на test, 5000 пользователей. CLF это классификатор LightGBM, RANK это ранкер LightGBM LambdaRank.

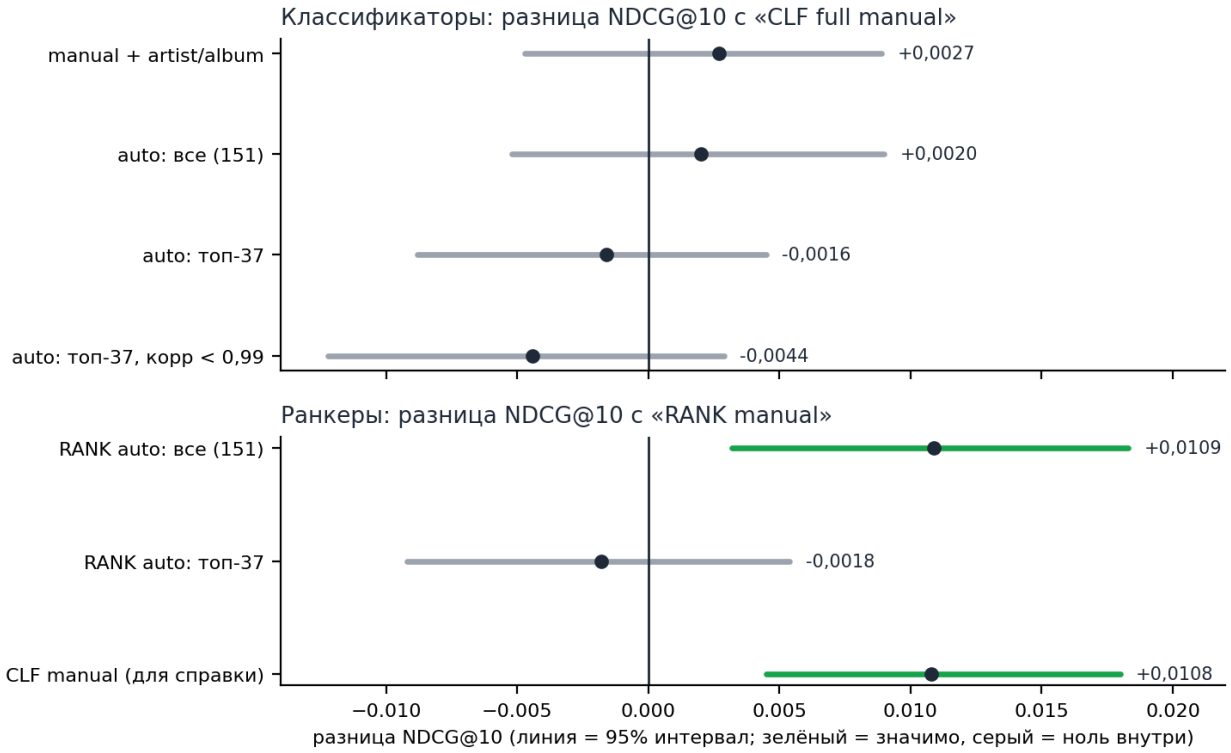
Модель	Призн.	NDCG@10 test	Recall@10 test	NDCG@10 val
random	—	0,0575	0,0806	0,0497
top-popular	—	0,0553	0,0806	0,0497
CLF minimal (только «всё время»)	10	0,0823	0,1128	0,0756
CLF full manual (ручная планка)	37	0,0818	0,1123	0,0722
CLF manual + artist/album	61	0,0852	0,1136	0,0777
CLF auto: все признаки	151	0,0848	0,1151	0,0812
CLF auto: топ-37	37	0,0785	0,1077	0,0700
CLF auto: топ-37, корр < 0,99	37	0,0780	0,1036	0,0767
CLF auto: топ-37, корр < 0,95	37	0,0775	0,1066	0,0742
CLF auto: топ-37, корр < 0,90	37	0,0832	0,1109	0,0752
RANK manual	37	0,0721	0,0949	0,0689
RANK auto: все признаки	151	0,0825	0,1140	0,0796
RANK auto: топ-37	37	0,0691	0,0960	0,0642



Порог отсева по корреляции выбирался по val: выбран 0,99. Вариант 0,90 на тесте случайно получился лучше, но выбирать его после просмотра теста было бы подгонкой, поэтому он в сравнение не взят.

Значимость различий (bootstrap)

На графике каждая линия это 95%-интервал разницы NDCG@10 с базовой моделью. Если линия пересекает вертикальный ноль, разницу нельзя отличить от шума (серый цвет). Если вся линия по одну сторону от нуля, разница настоящая (зелёный).



9. Что это значит: выводы человеческим языком

9.1. Что подтвердилось

- **Модель с признаками работает.** Любой бустинг с признаками примерно на 40% лучше случайного порядка по NDCG@10 (0,082 против 0,0575) и на 40–45% по Recall@10 (около 0,112 против 0,081).
- **Автоматика нашла то, что человек пропустил.** Самый важный признак во всех экспериментах это доля лайков юзера за 180 дней (`u_cr_180d`, его важность втрое выше, чем у следующего за ним). В ручном наборе окон длиннее 30 дней не было. Из 37 лучших автоматических признаков 21 использует окно от 60 дней или «всё время», а окна 1, 3 и 7 дней занимают всего 8 мест. Это прямой ответ на вопрос задания: окна, выбранные человеком, были слишком короткими.
- **Ранкер сдвинул отбор.** В отборе классификатора было 27 признаков юзера и 6 признаков трека, в отборе ранкера 17 и 15. Ранкер больше опирается на признаки, различающие треки между собой, как и ожидалось.

9.2. Что не подтвердилось

- **Автоматика не обыграла ручной набор.** Во всех сравнениях классификаторов интервалы включают ноль. Лучшая точечная разница +0,0027 по NDCG, но шум в два с лишним раза больше.
- **Отбор 37 лучших по важности не победил ручные 37** при равном бюджете признаков: NDCG test 0,0785 против 0,0818. Диагноз «в отборе много дублей» мы проверили через отсев по корреляции, и он не помог.
- **Победа ранкера кажущаяся.** «RANK auto: все» значимо лучше «RANK manual» (+0,0109, интервал [+0,003; +0,018]). Но «RANK manual» сам слабее классификатора на тех же признаках на ту же величину (+0,0108), так что относительно лучшей ручной модели выигрыша нет. Это классическая ловушка слабой базы: выиграть у слабого соперника легко.

> **Честная формулировка результата для отчёта:** автоматический генератор признаков не хуже ручного набора по качеству ранжирования и самостоятельно обнаруживает более длинные окна, чем выбрал человек. Превосходство над затюненным ручным бустингом на этих данных и в этом объёме не показано.

10. Ограничения: чего мы не знаем

- **Шум.** Интервалы около ±0,007 по NDCG@10. Различия меньше этого не подтверждены.
- **Один сид и одна выборка.** 5000 пользователей, выбор зафиксирован. На другой выборке цифры могут сдвинуться.
- **Нет тюнинга.** Гиперпараметры бустинга и ранкера не подбирались (Optuna не запускали). Значит, «хорошо затюненный бустинг» мы не побеждали и не проверяли.
- **Test смотрели много раз.** Варианты выбирались по val, но отдельного «чистого» test у нас нет.
- **Кандидаты это только прослушанные треки.** Это упрощение: настоящая рекомендация выбирает из всего каталога.
- **Метка условная.** «Лайк в течение часа» выбран нами, и другие горизонты не проверялись.
- **Масштаб.** Всё проверено на подвыборке. На миллиардах взаимодействий подход не запускали.

11. Что можно сделать дальше

Шаг	Зачем	Сложность
Bootstrap ранкеров против «CLF manual»	Закрыть ловушку слабой базы: сравнить с лучшей ручной моделью	5 минут
Optuna для ручной и автоматической моделей	Честно затюнить «противника» и сравнить на равных	1–2 часа
Больше пользователей и несколько сидов	Сузить интервалы, чтобы малые различия стали видны	до часа на прогон
Новые агрегаты (давность последнего события, затухание)	Счётчики по окнам не различают свежие и старые события внутри окна	1 день
LLM-агент пишет SQL-признаки	Идея из задания: пусть агент сам предлагает шаблоны признаков	1–2 дня

12. Репозиторий и как в нём ориентироваться

Весь код лежит на GitHub: `github.com/w4ngle/recsys-auto-features`. Главная страница (README) содержит краткие результаты, а этот PDF полное объяснение.

Файл	Что делает
`src/data.py`	загрузка данных, выбор 5000 пользователей, метки, деление по времени
`src/features.py`	ручные признаки и сборка обучающей выборки из 4 срезов
`src/auto_features.py`	автоматический генератор признаков по шаблону
`src/metrics.py`	AUC, NDCG@10, Recall@10 и парный bootstrap
`src/models.py`	обучение LightGBM
`notebooks/T-Lab.ipynb`	эксперименты: сборка таблиц, сравнение моделей, автоматика, ранкер
`reports/`	таблицы результатов (csv, json) и PDF-отчёты

Порядок запуска ноутбука

- Собрать таблицы признаков на 5000 юзерах и сохранить на диск (самая долгая часть).
- Сравнить ручные модели, посчитать bootstrap.
- Сгенерировать автоматические признаки, обучить модели, посмотреть отбор.
- Обучить ранкеры.
- Сохранить результаты в `reports/` и закоммитить.

Если ядро ноутбука перезапустилось, все переменные теряются. Таблицы признаков сохранены в `data/*.parquet`, поэтому восстановиться можно за минуту, не пересчитывая всё. Эти файлы не попадают в git: они слишком тяжёлые и закрыты в `.gitignore`.

13. Как рассказать за две минуты

> «Рекомендательные системы обычно строят на бустинге с признаками, которые придумывают руками. Мы проверили, можно ли придумывать их автоматически. Взяли данные Яндекс.Музыки, 5000 пользователей, предсказывали, какие из прослушанных треков человек лайкнет. Собрали сильную модель с ручными признаками как планку. Дальше написали генератор: он перебирает комбинации «сущность × событие × окно» и создаёт 151 признак вместо 37. Выяснилось, что самые полезные признаки считаются по окну в 180 дней, а в ручном наборе окон длиннее 30 дней не было, то есть автоматика нашла то, что человек упустил. По качеству ранжирования автоматика не хуже ручной модели, но и не лучше: разницу в пределах шума мы измерили через bootstrap. Честное ограничение: мы не подбирали гиперпараметры, поэтому нельзя сказать, что автоматика обыграла затюненный бустинг. Следующий шаг это Optuna и больше данных, чтобы сузить шум.»

Вопросы, которые могут задать, и ответы

Вопрос	Ответ
Почему автоматика не победила?	Классификатор получает большую часть сигнала из признаков юзера, а они одинаковы у всех кандидатов юзера и порядок треков не меняют. Окна влияют слабо. Это наша гипотеза, а не доказанный факт.
Откуда вы знаете, что различия шум?	Через bootstrap: пересэмплировали пользователей 100 раз и посмотрели интервал разницы. Ноль внутри значит «не доказано».
Почему ранкер «выиграл»?	Только у слабой базы: ранкер на ручных признаках хуже классификатора на тех же признаках. Относительно лучшей ручной модели выигрыша нет.
Была ли утечка из будущего?	Мы защищались: деление по времени, признаки только до среза, `assert` в коде, фильтр уже лайкнутых пар.
Что бы вы сделали дальше?	Optuna для обеих сторон, больше пользователей и сидов, новые агрегаты (давность, затухание), агент для генерации SQL-признаков.